

Desarrollo de un sistema cognitivo usando MCP y bases de datos

Ricardo Rueda

Corporación Universitaria Iberoamericana

Computación Cognitiva para Big Data

Docente: Joaquin Sánchez

7 de junio de 2026

Desarrollo de un sistema cognitivo usando MCP y bases de datos

Resumen

El presente informe describe el diseño, la implementación y la validación de un servidor basado en Model Context Protocol (MCP), desarrollado en Python y conectado con una base de datos SQLite para la gestión de un inventario. El sistema incorpora operaciones CRUD y herramientas analíticas que permiten crear, consultar, actualizar, eliminar y listar productos, así como calcular el valor total del inventario, identificar productos agotados, determinar el producto más costoso y generar estadísticas generales. El proyecto se organizó mediante un entorno virtual, control de versiones con Git y publicación en GitHub. Las pruebas funcionales evidencian que el servidor y las consultas sobre la base de datos operan correctamente. Finalmente, se analiza el papel de MCP en los sistemas cognitivos, las ventajas de conectar modelos de lenguaje con herramientas externas, las posibilidades de escalamiento hacia Big Data y los riesgos asociados con el acceso y la modificación de datos por sistemas de inteligencia artificial.

Palabras clave: Model Context Protocol, sistemas cognitivos, SQLite, operaciones CRUD, Big Data, FastMCP.

Introducción

Las organizaciones generan volúmenes crecientes de información que deben ser almacenados, consultados y transformados en conocimiento útil. La gestión de grandes volúmenes de datos requiere mecanismos de organización, procesamiento y explotación que permitan apoyar la toma de decisiones (García-Alsina, 2017). En este contexto, los sistemas cognitivos pueden ampliar sus capacidades cuando se conectan con herramientas externas,

debido a que dejan de depender exclusivamente del conocimiento interno de un modelo y pueden interactuar con fuentes reales de información.

El Model Context Protocol proporciona una arquitectura para exponer herramientas, datos y servicios a clientes compatibles. En el proyecto desarrollado, MCP se utilizó para registrar funciones de Python capaces de interactuar con una base de datos SQLite. De esta manera, las operaciones sobre el inventario se estructuraron como herramientas disponibles para un sistema cognitivo.

El uso de bases de datos facilita la consulta y reutilización de información estructurada. Iniciativas de acceso a datos, como las bases y diccionarios publicados por el ICFES, muestran la importancia de documentar y organizar los datos para que puedan ser interpretados y utilizados correctamente (Instituto Colombiano para la Evaluación de la Educación [ICFES], 2021). Este informe presenta el proceso técnico seguido, los resultados de las pruebas y una reflexión sobre el potencial y los riesgos de esta integración.

Objetivos

Objetivo general

Diseñar e implementar un servidor MCP en Python que permita realizar operaciones CRUD y consultas analíticas sobre una base de datos de inventario, con el propósito de comprender cómo un sistema cognitivo puede interactuar con herramientas externas.

Objetivos específicos

- Crear una base de datos SQLite con una tabla para almacenar productos.
- Implementar herramientas MCP para crear, consultar, actualizar, eliminar y listar productos.

- Desarrollar consultas analíticas sobre el valor total, los productos agotados, el producto más costoso y las estadísticas generales.
- Validar el funcionamiento del sistema mediante pruebas organizadas y evidencias de ejecución.
- Documentar el proyecto y publicar el código fuente en un repositorio de GitHub.

Marco conceptual

Sistemas cognitivos y transformación digital

Los sistemas cognitivos integran capacidades de análisis, aprendizaje, comprensión del lenguaje e interacción con fuentes externas. Su aplicación se relaciona con procesos de transformación digital en los que las organizaciones incorporan datos y tecnologías inteligentes para mejorar sus operaciones y servicios. La experiencia presentada por CENTRUM PUCP (2017) sobre la transformación digital en la era cognitiva evidencia la relevancia estratégica de estas tecnologías en los entornos empresariales.

Model Context Protocol

MCP funciona como una capa de comunicación entre un cliente basado en modelos de lenguaje y las herramientas que pueden ejecutar acciones. En este proyecto, cada función marcada con el decorador `@mcp.tool()` se convirtió en una herramienta del servidor. El servidor se ejecutó mediante FastMCP y utilizó el transporte stdio, adecuado para la comunicación entre procesos locales.

Bases de datos y fuentes externas

Una base de datos permite almacenar información de forma estructurada, conservarla entre ejecuciones y realizar consultas mediante un lenguaje especializado. Además, las fuentes

externas pueden abarcar repositorios públicos y servicios especializados. Ogimet representa un ejemplo de acceso a datos meteorológicos (Ballester, 2021), mientras que Datos Abiertos Bogotá permite consultar conjuntos de datos públicos de diferentes sectores (Alcaldía Mayor de Bogotá, s. f.). Estas fuentes ilustran cómo un sistema cognitivo puede enriquecerse al integrar información procedente de plataformas externas.

Operaciones CRUD

CRUD corresponde a las operaciones Create, Read, Update y Delete. Estas acciones constituyen el núcleo de numerosos sistemas de información. En el inventario implementado, la creación permitió registrar productos; la lectura se utilizó para consultar y listar datos; la actualización modificó la cantidad disponible; y la eliminación retiró productos de la base de datos.

Metodología de desarrollo

El proyecto se desarrolló mediante una metodología incremental. Cada componente fue implementado, probado y almacenado en GitHub antes de continuar con la siguiente etapa. Este procedimiento facilitó la detección de errores y mantuvo un historial verificable de los cambios.

1. Preparación del entorno. Se verificaron Python, Git y Visual Studio Code. Luego se creó la carpeta del proyecto y un entorno virtual de Python.

2. Control de versiones. Se inicializó un repositorio local de Git, se creó el archivo `.gitignore` y se conectó el proyecto con un repositorio remoto de GitHub.

3. Base de datos. Se creó `database.py` con la función `init_db()`, encargada de generar `inventory.db` y la tabla `productos`.

4. Servidor MCP. Se creó `server.py`, se inicializó FastMCP con el nombre `InventarioDB` y se definió una función reutilizable de conexión con SQLite.

5. Herramientas CRUD. Se implementaron y probaron las funciones para crear, consultar, actualizar, eliminar y listar productos.

6. Herramientas analíticas. Se incorporaron consultas para calcular el valor total, buscar productos agotados, identificar el producto más costoso y generar estadísticas.

7. Pruebas y documentación. Se creó test_db.py, se organizaron evidencias y se documentó el proyecto mediante README.md.

Arquitectura de la solución

La solución se organizó en tres componentes principales: el servidor MCP, las funciones de acceso a datos y la base de datos SQLite. El cliente MCP puede solicitar la ejecución de una herramienta; FastMCP identifica la función correspondiente; y la función ejecuta una consulta parametrizada sobre SQLite.

Componente	Archivo o tecnología	Responsabilidad
Base de datos	SQLite / inventory.db	Almacenar los productos de forma persistente.
Inicialización	database.py	Crear la tabla productos si aún no existe.
Servidor	server.py / FastMCP	Registrar las herramientas MCP y ejecutar las consultas.
Pruebas	test_db.py	Validar las operaciones CRUD y las herramientas analíticas.
Documentación	README.md / GitHub	Explicar la instalación, ejecución y funcionamiento.

Implementación de la base de datos

La base de datos se creó con SQLite, debido a que no requiere un servidor independiente y permite almacenar información en un archivo local. La tabla productos incluyó un identificador autoincremental, nombre, categoría, cantidad y precio.

```
CREATE TABLE IF NOT EXISTS productos (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  nombre TEXT NOT NULL,
  categoria TEXT,
  cantidad INTEGER NOT NULL,
  precio REAL NOT NULL
)
```

Implementación del servidor MCP

El servidor se creó con FastMCP y recibió el nombre InventarioDB. Al iniciar server.py se ejecuta `init_db()`, lo que garantiza la existencia de la base de datos y de la tabla.

Posteriormente, las funciones se registran como herramientas mediante `@mcp.tool()`.

```
from fastmcp import FastMCP
from database import init_db, DB_NAME

init_db()
mcp = FastMCP("InventarioDB")
```

Operaciones CRUD implementadas

Herramienta	Operación	Resultado esperado
<code>crear_producto</code>	Inserta un producto.	Mensaje de creación exitosa.
<code>consultar_producto</code>	Busca un producto por ID.	Diccionario del producto o mensaje de error.
<code>actualizar_producto</code>	Modifica la cantidad.	Confirmación de actualización.
<code>eliminar_producto</code>	Elimina un registro por ID.	Confirmación o producto no encontrado.
<code>listar_productos</code>	Recupera todos los registros.	Lista de diccionarios.

Herramientas analíticas implementadas

calcular_valor_total_inventario: Suma el producto de cantidad por precio para todos los registros.

productos_agotados: Selecciona los productos cuya cantidad es igual a cero.

producto_mas_costoso: Ordena los productos por precio descendente y retorna el primero.

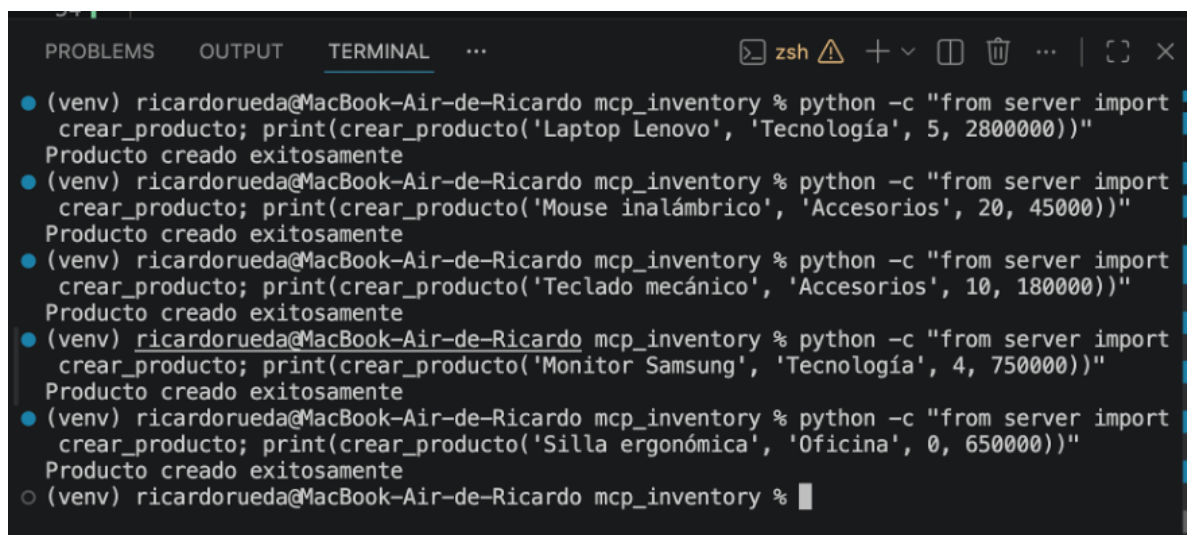
estadisticas_inventario: Calcula el número de productos, los promedios de cantidad y precio, y el valor total.

Resultados y evidencias

Las pruebas se realizaron con cinco productos. Posteriormente, se actualizó la cantidad del mouse inalámbrico y se eliminó el teclado mecánico. Las siguientes figuras presentan las evidencias principales de la ejecución.

Figura 1

Creación de los cinco productos de prueba



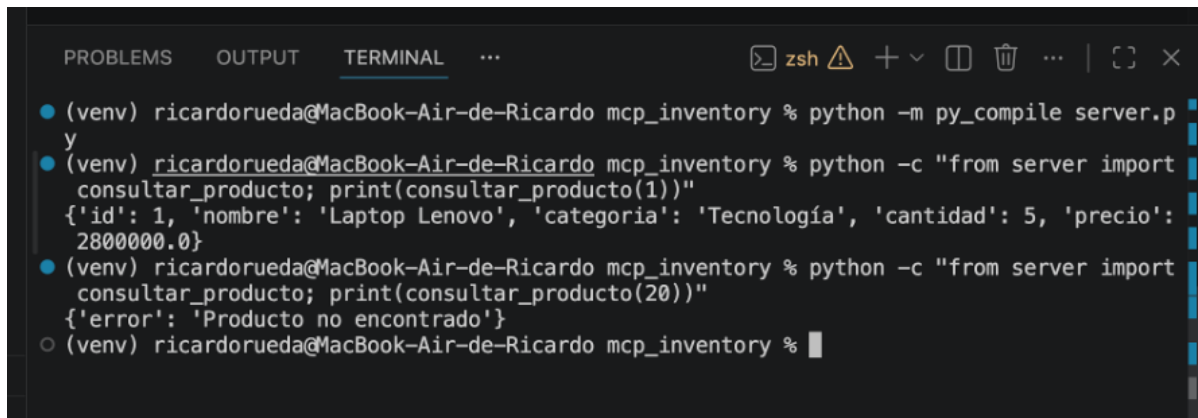
```
PROBLEMS  OUTPUT  TERMINAL  ...
zsh

● (venv) ricardorueda@MacBook-Air-de-Ricardo mcp_inventory % python -c "from server import
  crear_producto; print(crear_producto('Laptop Lenovo', 'Tecnología', 5, 280000))"
  Producto creado exitosamente
● (venv) ricardorueda@MacBook-Air-de-Ricardo mcp_inventory % python -c "from server import
  crear_producto; print(crear_producto('Mouse inalámbrico', 'Accesorios', 20, 45000))"
  Producto creado exitosamente
● (venv) ricardorueda@MacBook-Air-de-Ricardo mcp_inventory % python -c "from server import
  crear_producto; print(crear_producto('Teclado mecánico', 'Accesorios', 10, 180000))"
  Producto creado exitosamente
● (venv) ricardorueda@MacBook-Air-de-Ricardo mcp_inventory % python -c "from server import
  crear_producto; print(crear_producto('Monitor Samsung', 'Tecnología', 4, 750000))"
  Producto creado exitosamente
● (venv) ricardorueda@MacBook-Air-de-Ricardo mcp_inventory % python -c "from server import
  crear_producto; print(crear_producto('Silla ergonómica', 'Oficina', 0, 650000))"
  Producto creado exitosamente
○ (venv) ricardorueda@MacBook-Air-de-Ricardo mcp_inventory %
```

Nota. La terminal muestra la ejecución de `crear_producto` y la confirmación de cada inserción.

Figura 2

Consulta de un producto existente y uno inexistente



```

PROBLEMS  OUTPUT  TERMINAL  ...
zsh

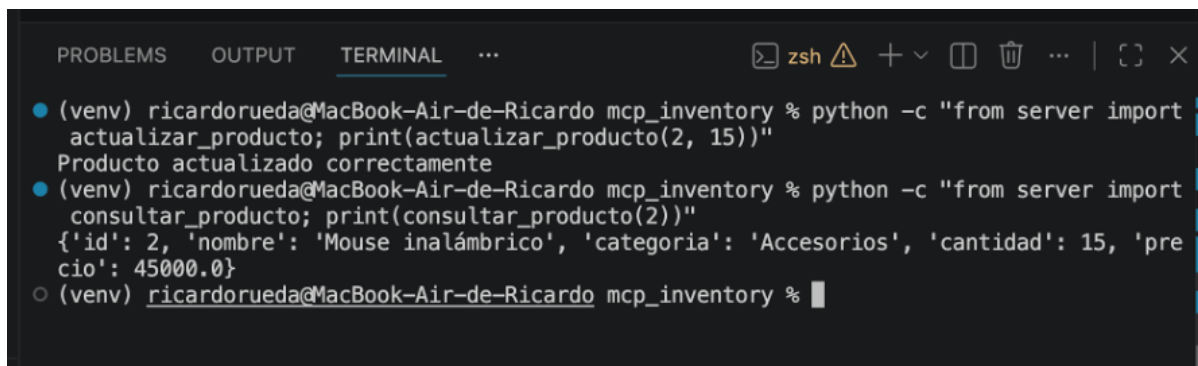
• (venv) ricardorueda@MacBook-Air-de-Ricardo mcp_inventory % python -m py_compile server.py
• (venv) ricardorueda@MacBook-Air-de-Ricardo mcp_inventory % python -c "from server import
consultar_producto; print(consultar_producto(1))"
{'id': 1, 'nombre': 'Laptop Lenovo', 'categoria': 'Tecnología', 'cantidad': 5, 'precio':
2800000.0}
• (venv) ricardorueda@MacBook-Air-de-Ricardo mcp_inventory % python -c "from server import
consultar_producto; print(consultar_producto(20))"
{'error': 'Producto no encontrado'}
○ (venv) ricardorueda@MacBook-Air-de-Ricardo mcp_inventory %

```

Nota. Se observa el producto con ID 1 y el manejo del error para un identificador no registrado.

Figura 3

Actualización de la cantidad de un producto



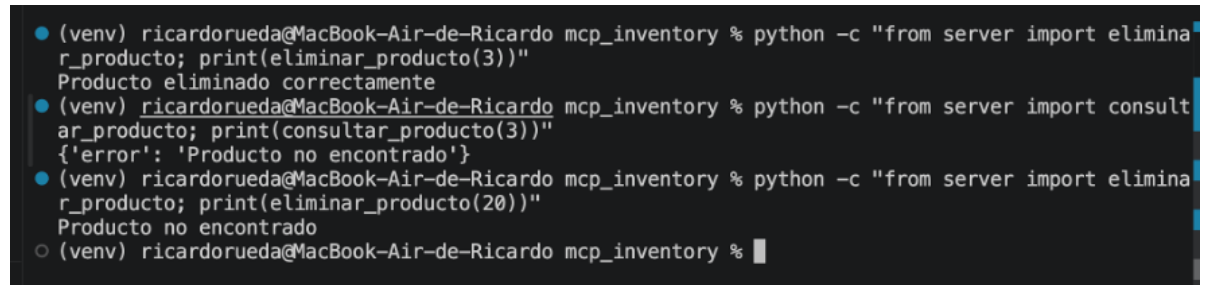
```

PROBLEMS  OUTPUT  TERMINAL  ...
zsh

• (venv) ricardorueda@MacBook-Air-de-Ricardo mcp_inventory % python -c "from server import
actualizar_producto; print(actualizar_producto(2, 15))"
Producto actualizado correctamente
• (venv) ricardorueda@MacBook-Air-de-Ricardo mcp_inventory % python -c "from server import
consultar_producto; print(consultar_producto(2))"
{'id': 2, 'nombre': 'Mouse inalámbrico', 'categoria': 'Accesorios', 'cantidad': 15, 'pre
cio': 45000.0}
○ (venv) ricardorueda@MacBook-Air-de-Ricardo mcp_inventory %

```

Nota. La cantidad del mouse inalámbrico se actualizó a 15 y fue verificada mediante una consulta posterior.

Figura 4*Eliminación y verificación de un producto*

```
(venv) ricardorueda@MacBook-Air-de-Ricardo mcp_inventory % python -c "from server import eliminar_producto; print(eliminar_producto(3))"
Producto eliminado correctamente
(venv) ricardorueda@MacBook-Air-de-Ricardo mcp_inventory % python -c "from server import consultar_producto; print(consultar_producto(3))"
{'error': 'Producto no encontrado'}
(venv) ricardorueda@MacBook-Air-de-Ricardo mcp_inventory % python -c "from server import eliminar_producto; print(eliminar_producto(20))"
Producto no encontrado
(venv) ricardorueda@MacBook-Air-de-Ricardo mcp_inventory %
```

Nota. La consulta posterior confirma que el producto eliminado ya no se encuentra en la base de datos.

Figura 5*Listado general del inventario*

```

(venv) ricardorueda@MacBook-Air-de-Ricardo mcp_inventory % python -c "from server import listar_
productos; print(listar_productos())"
• (venv) ricardorueda@MacBook-Air-de-Ricardo mcp_inventory % clear
• (venv) ricardorueda@MacBook-Air-de-Ricardo mcp_inventory % python -c "from server import listar_
productos; from pprint import pprint; pprint(listar_productos(), sort_dicts=False)"
[{'id': 1,
  'nombre': 'Laptop Lenovo',
  'categoria': 'Tecnología',
  'cantidad': 5,
  'precio': 280000.0},
 {'id': 2,
  'nombre': 'Mouse inalámbrico',
  'categoria': 'Accesorios',
  'cantidad': 15,
  'precio': 45000.0},
 {'id': 4,
  'nombre': 'Monitor Samsung',
  'categoria': 'Tecnología',
  'cantidad': 4,
  'precio': 75000.0},
 {'id': 5,
  'nombre': 'Silla ergonómica',
  'categoria': 'Oficina',
  'cantidad': 0,
  'precio': 65000.0}]
○ (venv) ricardorueda@MacBook-Air-de-Ricardo mcp_inventory %

```

Nota. La salida utiliza pprint para mostrar los productos restantes de forma legible.

Figura 6*Cálculo del valor total del inventario*

```

• (venv) ricardorueda@MacBook-Air-de-Ricardo mcp_inventory % python -m py_compile server.py
• (venv) ricardorueda@MacBook-Air-de-Ricardo mcp_inventory % python -c "from server import calcula
r_valor_total_inventario; print(calcular_valor_total_inventario())"
{'valor_total_inventario': 1767500.0}
○ (venv) ricardorueda@MacBook-Air-de-Ricardo mcp_inventory %

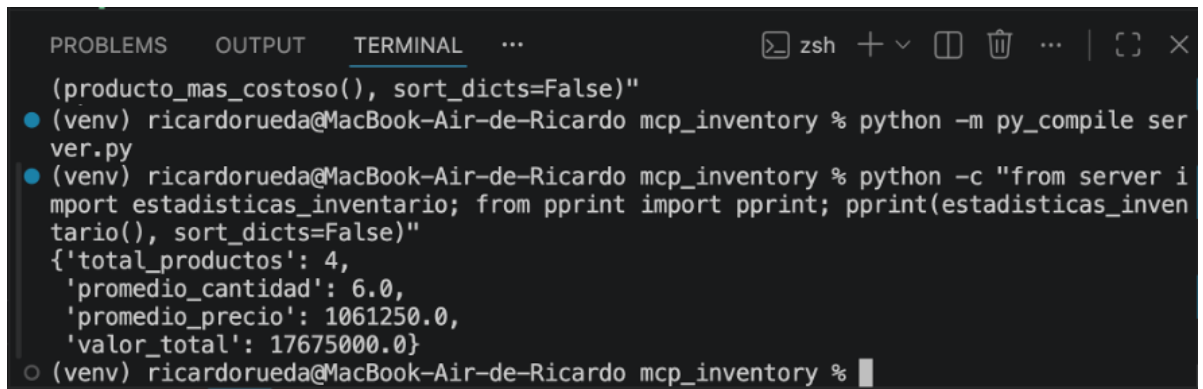
```

Nota. La herramienta retorna el valor económico acumulado del inventario.

Figura 7*Consulta de productos agotados*

```
(venv) ricardorueda@MacBook-Air-de-Ricardo mcp_inventory % python -m py_compile server.py
(venv) ricardorueda@MacBook-Air-de-Ricardo mcp_inventory % python -c "from server import products_agotados; from pprint import pprint; pprint(products_agotados(), sort_dicts=False)"
[{'id': 5,
  'nombre': 'Silla ergonómica',
  'categoria': 'Oficina',
  'cantidad': 0,
  'precio': 650000.0}]
(venv) ricardorueda@MacBook-Air-de-Ricardo mcp_inventory %
```

Nota. La silla ergonómica se identifica con una cantidad igual a cero.

Figura 8*Identificación del producto más costoso*


```

PROBLEMS  OUTPUT  TERMINAL  ...
(producto_mas_costoso(), sort_dicts=False)"
● (venv) ricardorueda@MacBook-Air-de-Ricardo mcp_inventory % python -m py_compile server.py
● (venv) ricardorueda@MacBook-Air-de-Ricardo mcp_inventory % python -c "from server import estadisticas_inventario; from pprint import pprint; pprint(estadisticas_inventario(), sort_dicts=False)"
{'total_productos': 4,
 'promedio_cantidad': 6.0,
 'promedio_precio': 1061250.0,
 'valor_total': 17675000.0}
○ (venv) ricardorueda@MacBook-Air-de-Ricardo mcp_inventory %

```

Nota. La Laptop Lenovo presenta el precio unitario más alto.

Figura 9*Estadísticas generales del inventario*


```

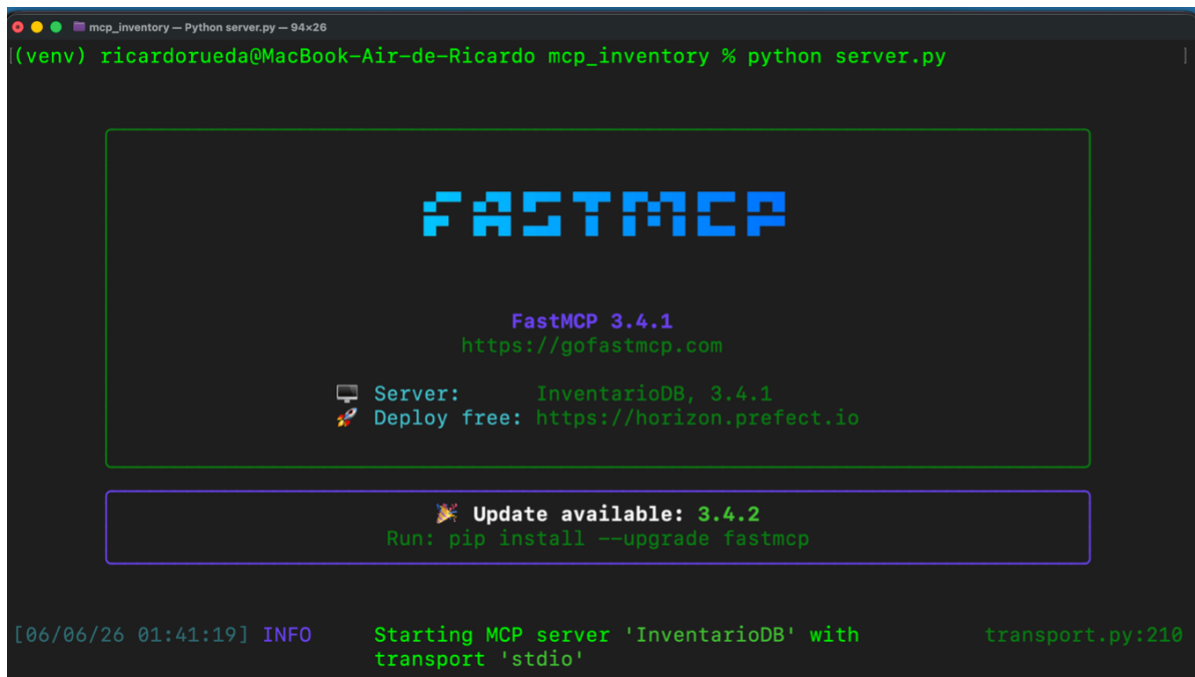
● (venv) ricardorueda@MacBook-Air-de-Ricardo mcp_inventory % python -m py_compile server.py
● (venv) ricardorueda@MacBook-Air-de-Ricardo mcp_inventory % python -c "from server import producto_mas_costoso; from pprint import pprint; pprint(producto_mas_costoso(), sort_dicts=False)"
{'id': 1,
 'nombre': 'Laptop Lenovo',
 'categoria': 'Tecnologia',
 'cantidad': 5,
 'precio': 2800000.0}
○ (venv) ricardorueda@MacBook-Air-de-Ricardo mcp_inventory %

```

Nota. Se muestran el total de productos, los promedios y el valor total.

Figura 10

Ejecución formal del servidor MCP



The screenshot shows a terminal window titled "mcp_inventory - Python server.py - 94x26". The command prompt shows the user "ricardorueda@MacBook-Air-de-Ricardo" in a virtual environment "mcp_inventory" running the command "python server.py". The output displays the "FASTMCP" logo in blue, followed by "FastMCP 3.4.1" and the website "https://gofastmcp.com". Below this, it shows "Server: InventarioDB, 3.4.1" and "Deploy free: https://horizon.prefect.io". A message indicates an update is available to version 3.4.2, with the command "Run: pip install --upgrade fastmcp". At the bottom, a log message states "[06/06/26 01:41:19] INFO Starting MCP server 'InventarioDB' with transport 'stdio'" and "transport.py:218".

```
mcp_inventory - Python server.py - 94x26
[(venv) ricardorueda@MacBook-Air-de-Ricardo mcp_inventory % python server.py

FASTMCP

FastMCP 3.4.1
https://gofastmcp.com

Server: InventarioDB, 3.4.1
Deploy free: https://horizon.prefect.io

Update available: 3.4.2
Run: pip install --upgrade fastmcp

[06/06/26 01:41:19] INFO Starting MCP server 'InventarioDB' with transport 'stdio' transport.py:218
```

Nota. FastMCP inicia el servidor InventarioDB mediante el transporte stdio.

Figura 11*Repositorio del proyecto en GitHub*

Nota. El repositorio contiene el código fuente, las pruebas, las dependencias y el archivo README.md.

Análisis de resultados

Las pruebas confirmaron que las operaciones CRUD funcionan sobre datos persistentes. La información continuó disponible incluso después de reiniciar el computador, lo que demuestra una diferencia esencial entre el almacenamiento en una base de datos y el mantenimiento temporal de variables en memoria.

Las consultas analíticas produjeron resultados coherentes con el estado del inventario. El valor total se calculó a partir de la multiplicación de la cantidad por el precio de cada producto.

La búsqueda de productos agotados identificó correctamente el registro con cantidad cero, mientras que la ordenación descendente permitió recuperar el producto más costoso.

Los datos urbanos y de servicios pueden aumentar considerablemente en volumen y complejidad. El análisis de ciudades inteligentes presentado por DW Español (2021) muestra tanto el potencial del uso masivo de datos como los desafíos de privacidad. Esta reflexión resulta aplicable al proyecto: aunque el inventario es pequeño, una ampliación hacia múltiples sedes o fuentes en tiempo real exigiría controles técnicos y éticos más robustos.

Preguntas de análisis

1. ¿Qué papel cumple MCP en la construcción de sistemas cognitivos?

El Model Context Protocol cumple la función de conectar un modelo de lenguaje con herramientas y fuentes de información externas, como bases de datos, archivos, APIs y servicios computacionales. Esta conexión permite que un sistema cognitivo no se limite únicamente a generar respuestas textuales a partir de la información aprendida durante su entrenamiento, sino que también pueda consultar datos reales, ejecutar funciones y realizar operaciones sobre sistemas externos.

En el proyecto desarrollado, MCP actúa como intermediario entre un posible modelo de lenguaje y la base de datos SQLite utilizada para almacenar la información del inventario. Las funciones implementadas en Python se registraron como herramientas del servidor MCP mediante el decorador `@mcp.tool()`. Entre estas herramientas se encuentran la creación, consulta, actualización, eliminación y listado de productos, además de varias funciones de análisis del inventario.

Por tanto, MCP amplía las capacidades de un sistema cognitivo al proporcionar una forma estructurada y controlada de interactuar con información externa. En lugar de depender

solamente del conocimiento interno del modelo, el sistema puede utilizar herramientas especializadas para obtener datos actualizados y ejecutar acciones concretas.

2. ¿Por qué es importante conectar un modelo de lenguaje con herramientas externas?

La conexión de un modelo de lenguaje con herramientas externas es importante porque estos modelos, por sí solos, no tienen acceso directo a bases de datos institucionales, archivos locales, sistemas empresariales ni información actualizada en tiempo real. Aunque pueden interpretar instrucciones y generar lenguaje natural, necesitan mecanismos adicionales para consultar datos específicos o ejecutar operaciones sobre sistemas reales.

Al utilizar herramientas externas, un modelo de lenguaje puede complementar sus capacidades de comprensión y generación con acciones concretas. Por ejemplo, puede consultar el inventario de una empresa, identificar productos agotados, calcular valores económicos o actualizar la cantidad disponible de un producto. Esto transforma al modelo en un componente capaz de apoyar tareas operativas y analíticas.

En este proyecto, la conexión con SQLite permitió que las herramientas del servidor MCP trabajaran sobre información almacenada de manera persistente. Gracias a esta integración, el sistema puede responder con datos obtenidos directamente de la base de datos y no con información supuesta o generada sin verificación. Esta integración se relaciona con los procesos de transformación digital y uso empresarial de tecnologías cognitivas descritos por CENTRUM PUCP (2017).

3. ¿Qué ventajas ofrece el uso de una base de datos frente al manejo de archivos planos?

Una base de datos ofrece ventajas importantes frente al uso de archivos planos, como archivos de texto o CSV. En primer lugar, permite almacenar la información de manera estructurada mediante tablas, columnas, tipos de datos y relaciones. Esto facilita la organización y reduce la posibilidad de inconsistencias.

Otra ventaja es la posibilidad de realizar consultas utilizando SQL. Por medio de este lenguaje se pueden buscar, filtrar, ordenar, actualizar y eliminar registros de forma eficiente. En el proyecto, las consultas SQL permitieron encontrar productos por su identificador, listar productos agotados, calcular el valor total del inventario y determinar el producto más costoso.

Las bases de datos también permiten mantener la persistencia de la información. Esto significa que los datos permanecen almacenados aunque el programa se cierre o el computador se reinicie. Además, una base de datos ofrece mejores posibilidades de integridad y escalabilidad. La publicación de bases de datos acompañadas de diccionarios, como en el caso del ICFES (2021), evidencia la importancia de estructurar y documentar adecuadamente la información para facilitar su interpretación y reutilización.

4. ¿Cómo podría ampliarse esta solución para un escenario de Big Data?

La solución desarrollada constituye un prototipo funcional orientado a un inventario pequeño almacenado en SQLite. Para utilizarla en un escenario de Big Data sería necesario modificar algunos componentes de la arquitectura.

En primer lugar, SQLite podría reemplazarse por una base de datos con mayor capacidad de almacenamiento, concurrencia y procesamiento, como PostgreSQL, MongoDB, Cassandra o una solución administrada en la nube. Si el sistema recibiera información de múltiples

almacenes, plataformas de comercio electrónico, sensores o sistemas logísticos, podría incorporarse una arquitectura distribuida. Apache Kafka permitiría recibir datos en tiempo real, mientras que Apache Spark podría utilizarse para procesar grandes volúmenes de información de forma paralela.

También sería posible desplegar el servidor MCP y la base de datos en plataformas cloud, construir un data lake o lakehouse e incorporar dashboards y modelos predictivos. La diversidad de fuentes disponibles, como los datos meteorológicos de Ogimet (Ballester, 2021) o los conjuntos de Datos Abiertos Bogotá (Alcaldía Mayor de Bogotá, s. f.), ejemplifica el tipo de información externa que podría integrarse. La gestión y explotación de grandes volúmenes de datos exige arquitecturas que respondan al volumen, la velocidad y la variedad de la información (García-Alsina, 2017).

5. ¿Qué riesgos se deben considerar al permitir que un modelo de IA consulte o modifique datos?

Permitir que un modelo de inteligencia artificial consulte o modifique datos genera riesgos que deben ser considerados durante el diseño del sistema. Uno de los principales es el acceso no autorizado. Si no se implementan mecanismos de autenticación y permisos, un usuario podría consultar información confidencial o ejecutar operaciones que no le corresponden.

Otro riesgo es la modificación o eliminación accidental de información. Un modelo podría interpretar de manera incorrecta una instrucción y actualizar o borrar registros que no debían modificarse. También se debe considerar la calidad de los datos ingresados. En una aplicación real se deberían validar condiciones como cantidades no negativas, precios válidos, campos obligatorios y tipos de datos correctos.

La inyección SQL representa otro riesgo cuando las consultas se construyen concatenando directamente los valores proporcionados por el usuario. En el proyecto se utilizaron consultas parametrizadas, lo que contribuye a reducir este riesgo. Además, existe la posibilidad de exposición de datos sensibles y de vigilancia excesiva. El caso de las ciudades basadas en Big Data muestra que la capacidad de recopilar y relacionar datos puede generar preocupaciones sobre privacidad y control (DW Español, 2021).

Para reducir estos riesgos, es recomendable utilizar autenticación, autorización por roles, validaciones de entrada, copias de seguridad y registros de auditoría. También se pueden establecer límites sobre las herramientas disponibles para el modelo y solicitar confirmación antes de ejecutar operaciones críticas.

Conclusiones

El desarrollo del servidor MCP permitió comprobar que un sistema cognitivo puede ampliar sus capacidades cuando se conecta con herramientas externas. Las funciones registradas mediante FastMCP actuaron como una interfaz estructurada para consultar y modificar información almacenada en SQLite.

La implementación incremental y el uso de Git facilitaron la validación de cada componente y mantuvieron un historial de cambios. El repositorio de GitHub centralizó el código, las instrucciones de instalación y las pruebas funcionales.

SQLite resultó apropiado para un prototipo académico por su facilidad de configuración y persistencia. Sin embargo, una aplicación con grandes volúmenes de datos requeriría bases de datos escalables, procesamiento distribuido, servicios cloud y controles de seguridad más avanzados.

La integración de inteligencia artificial con datos reales ofrece beneficios operativos y analíticos, pero requiere validación, autenticación, autorización, auditoría y protección de la privacidad. El acceso a herramientas debe diseñarse bajo el principio de mínimo privilegio.

Publicación del proyecto

El código fuente, el archivo README y el historial de desarrollo se encuentran disponibles en el repositorio:

https://github.com/rrueda0584-beep/mcp_inventory

Referencias

García-Alsina, M. (2017). Big data: Gestión y explotación de grandes volúmenes de datos.

Editorial UOC. <https://elibro.net/es/lc/biblioibero/titulos/116303>

Instituto Colombiano para la Evaluación de la Educación. (2021). Acceso a bases de datos y

diccionarios. <https://www.icfes.gov.co/investigadores-y-estudiantes-posgrado/acceso-a-bases-de-datos>

Ballester, G. (2021). Ogimet: Base de datos meteorológicos. <https://www.ogimet.com/>

Alcaldía Mayor de Bogotá. (s. f.). Datos Abiertos Bogotá. <https://datosabiertos.bogota.gov.co/>

CENTRUM PUCP. (2017). IBM Perú y su transformación digital en la era cognitiva – XII

Semana Internacional [Video]. YouTube.

<https://www.youtube.com/watch?v=yE7yNnRY1bI>

DW Español. (2021). La ciudad que lo sabe todo sobre ti | Big Data: ciudades del futuro

[Video]. YouTube.